

CI/CD Workflow Report

Pipeline architecture, stages, automation results and observations

1. Introduction

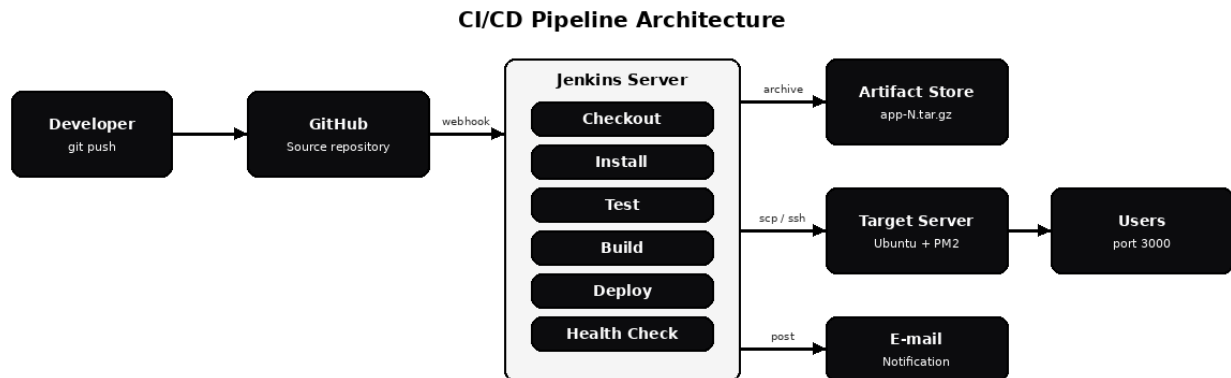
Continuous Integration and Continuous Deployment (CI/CD) is a set of practices that automatically validates, packages and releases every code change. This report documents a CI/CD workflow built with **GitHub**, **Jenkins** and a Linux **target server** for a Node.js (Express) application. It describes the architecture, each pipeline stage, the results of the automated runs and the observations made.

2. Objectives

- Design a pipeline that runs automatically on every push to the main branch.
- Automate testing, packaging and deployment with no manual steps.
- Verify each release using a health check and notify the team of the result.
- Measure build results and record observations to identify improvements.

3. Pipeline Architecture

A developer pushes code to GitHub. A webhook notifies Jenkins, which pulls the repository and runs the Jenkinsfile. After the tests pass, the application is packaged and copied to the target server over SSH, where PM2 restarts it. The artifact is archived and the team is notified by e-mail.



Push -> webhook -> automated stages -> deploy -> health check -> archive and notify

Figure 1: Architecture of the CI/CD pipeline

3.1 Components

Component	Role in the workflow
Developer workstation	Writes code and pushes commits to the main branch
GitHub repository	Stores source code and the Jenkinsfile; sends a webhook on every push
Jenkins server	Orchestrates the pipeline: checkout, test, build, deploy, notify
Artifact store	Jenkins archive holding the versioned package app-N.tar.gz
Target server	Ubuntu host running the application under PM2 on port 3000
E-mail (SMTP)	Sends success and failure notifications to the team

4. Pipeline Stages

The pipeline is defined as code in a declarative Jenkinsfile. It has six stages that run in order; if a stage fails, the remaining stages are skipped and the failure notification is sent.

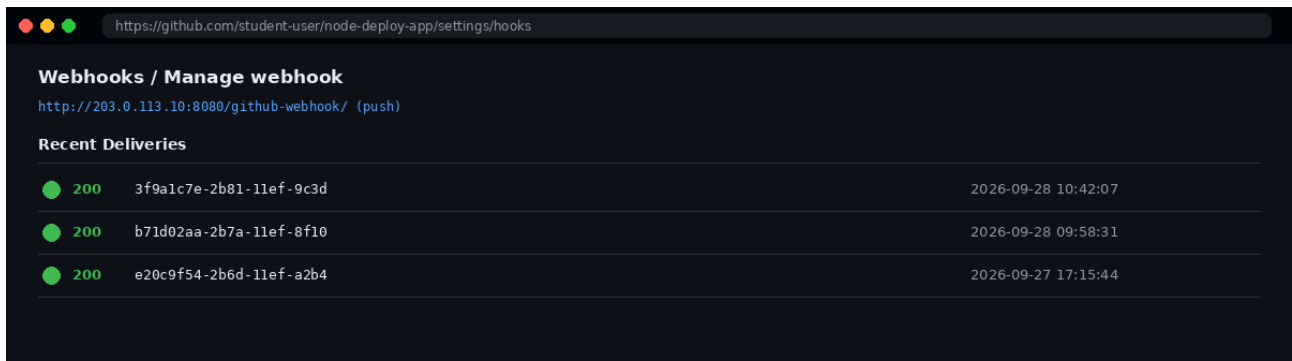
#	Stage	Action	Success criteria
1	Checkout	Clone the main branch from GitHub	Repository cloned
2	Install Dependencies	npm ci	Packages installed from lockfile
3	Test	npm test (Jest + Supertest)	All tests pass
4	Build	Create app-N.tar.gz package	Artifact created
5	Deploy	scp package, extract, pm2 restart over SSH	PM2 process online
6	Health Check	curl /health on target server	HTTP 200 and status UP

4.1 Trigger configuration

Three triggers are configured. The webhook gives instant feedback, Poll SCM acts as a fallback if a webhook is missed, and the nightly cron build detects environment problems even when no code changes.

Jenkinsfile (triggers)

```
triggers {
  githubPush()           // instant build on git push (webhook)
  pollSCM('H/5 * * * *') // fallback: check repository every 5 minutes
  cron('H 2 * * *')      // nightly build around 2 AM
}
```



Webhooks / Manage webhook

http://203.0.113.10:8080/github-webhook/ (push)

Recent Deliveries

200	3f9a1c7e-2b81-11ef-9c3d	2026-09-28 10:42:07
200	b71d02aa-2b7a-11ef-8f10	2026-09-28 09:58:31
200	e20c9f54-2b6d-11ef-a2b4	2026-09-27 17:15:44

Figure 2: GitHub webhook deliveries returning HTTP 200 from Jenkins

4.2 Deployment and post-build actions

Jenkinsfile (deploy and post)

```

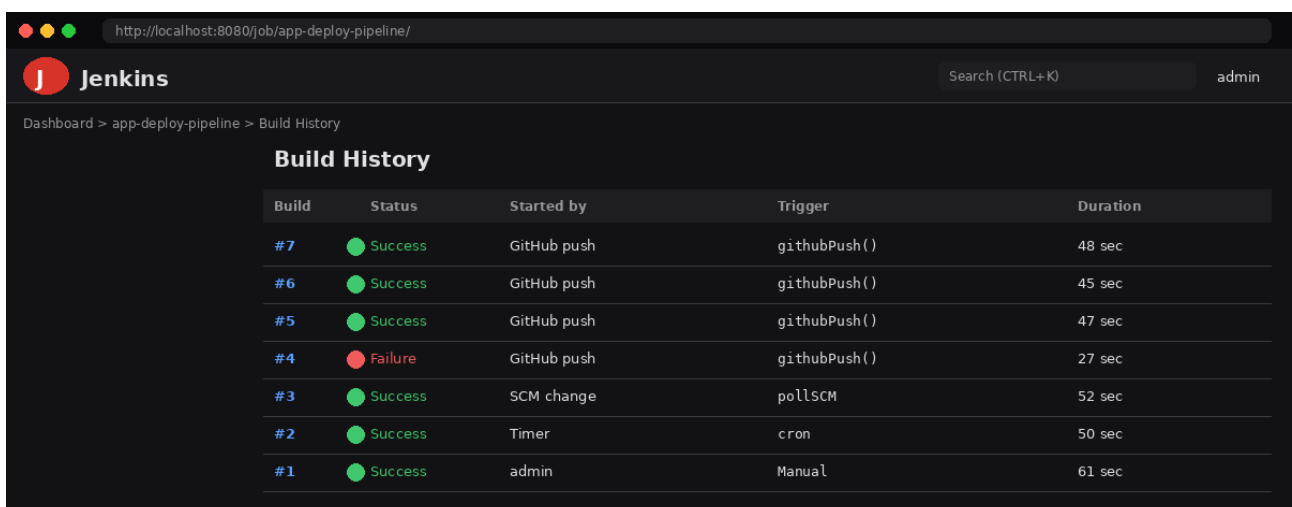
stage('Deploy') {
    steps {
        sshagent(credentials: ['deploy-server-key']) {
            sh """
                scp ${ARTIFACT} ubuntu@10.0.0.25:/opt/myapp/
                ssh ubuntu@10.0.0.25 'cd /opt/myapp && tar -xzf ${ARTIFACT} &&
                npm ci --omit=dev && pm2 restart myapp --update-env'
            """
        }
    }
}

post {
    success { archiveArtifacts artifacts: 'app-*.tar.gz', fingerprint: true }
    failure { mail to: 'team@example.com', subject: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}" }
    always { cleanWs() }
}

```

5. Automation Results

The pipeline ran seven times: one manual run, one scheduled run, one Poll SCM run and four webhook runs. Six builds succeeded and one failed at the Test stage.



Jenkins

Dashboard > app-deploy-pipeline > Build History

Build	Status	Started by	Trigger	Duration
#7	Success	GitHub push	githubPush()	48 sec
#6	Success	GitHub push	githubPush()	45 sec
#5	Success	GitHub push	githubPush()	47 sec
#4	Failure	GitHub push	githubPush()	27 sec
#3	Success	SCM change	pollSCM	52 sec
#2	Success	Timer	cron	50 sec
#1	Success	admin	Manual	61 sec

Figure 3: Build history of app-deploy-pipeline (builds #1 to #7)

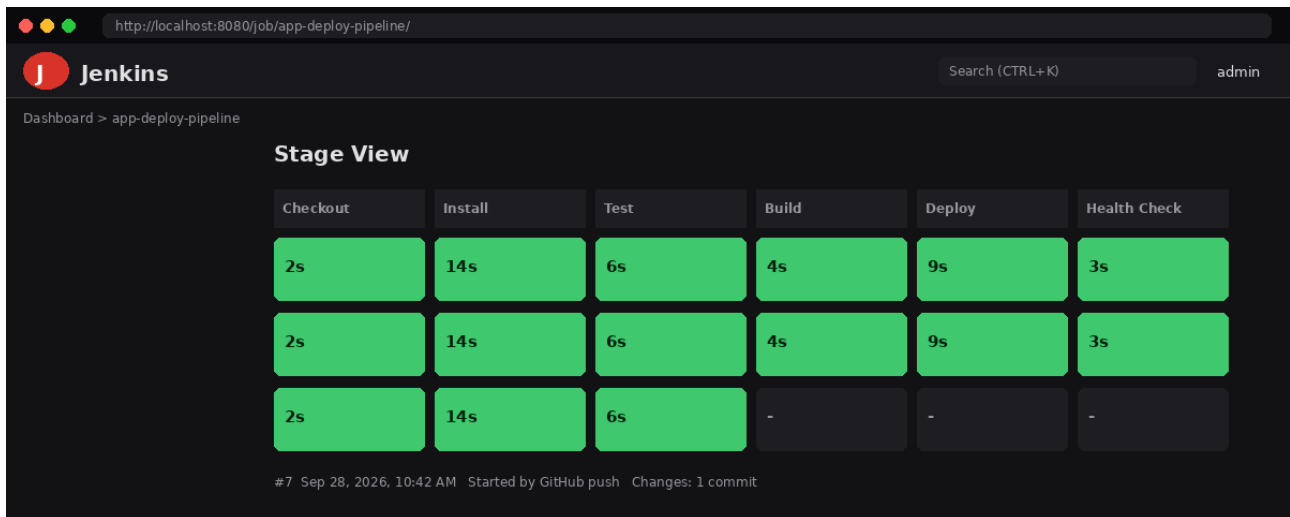


Figure 4: Stage View showing all six stages passing for the latest builds

5.1 Successful run (build #7)

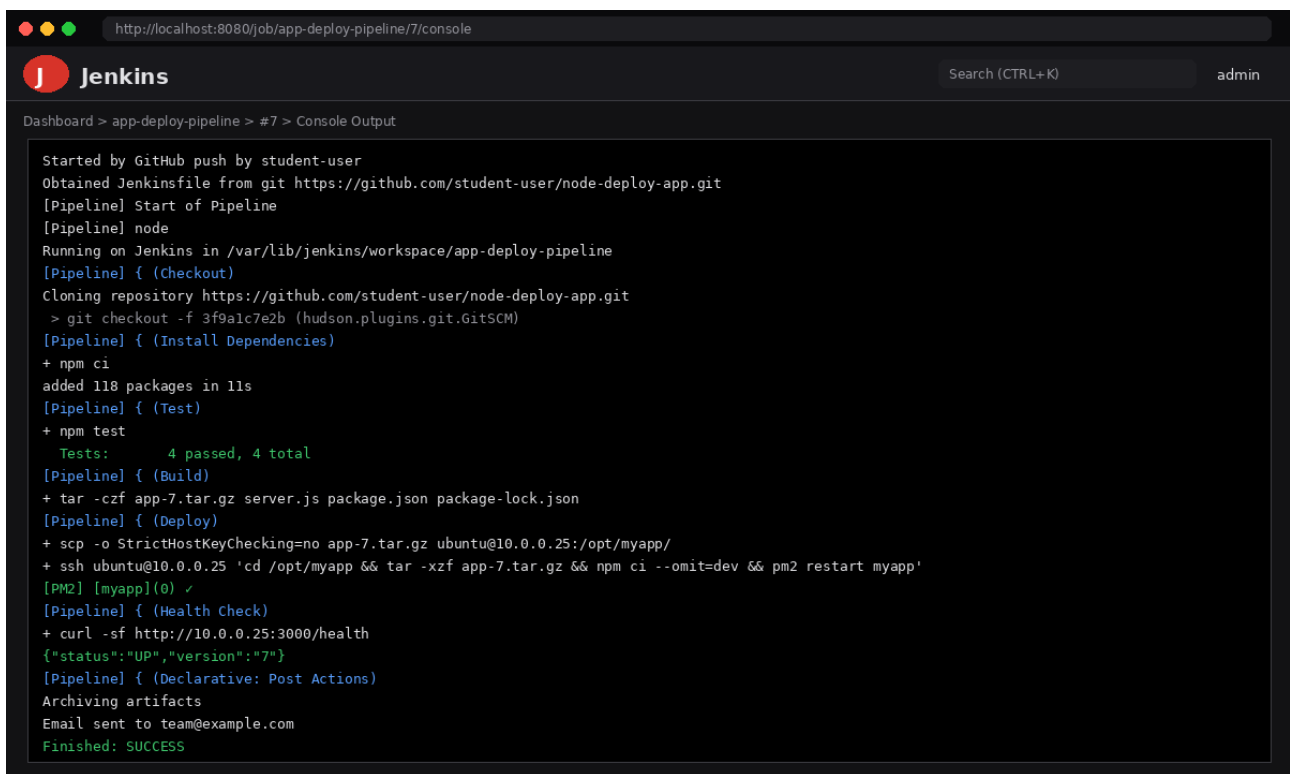
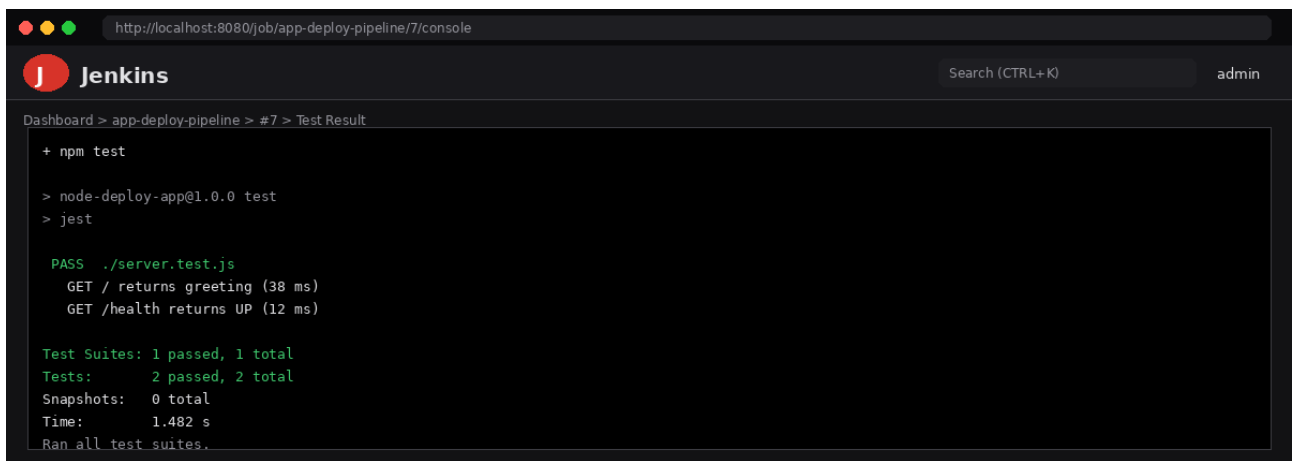


Figure 5: Console output of build #7 ending with Finished: SUCCESS



```
http://localhost:8080/job/app-deploy-pipeline/7/console
Jenkins
Search (CTRL+K) admin
Dashboard > app-deploy-pipeline > #7 > Test Result

+ npm test

> node-deploy-app@1.0.0 test
> jest

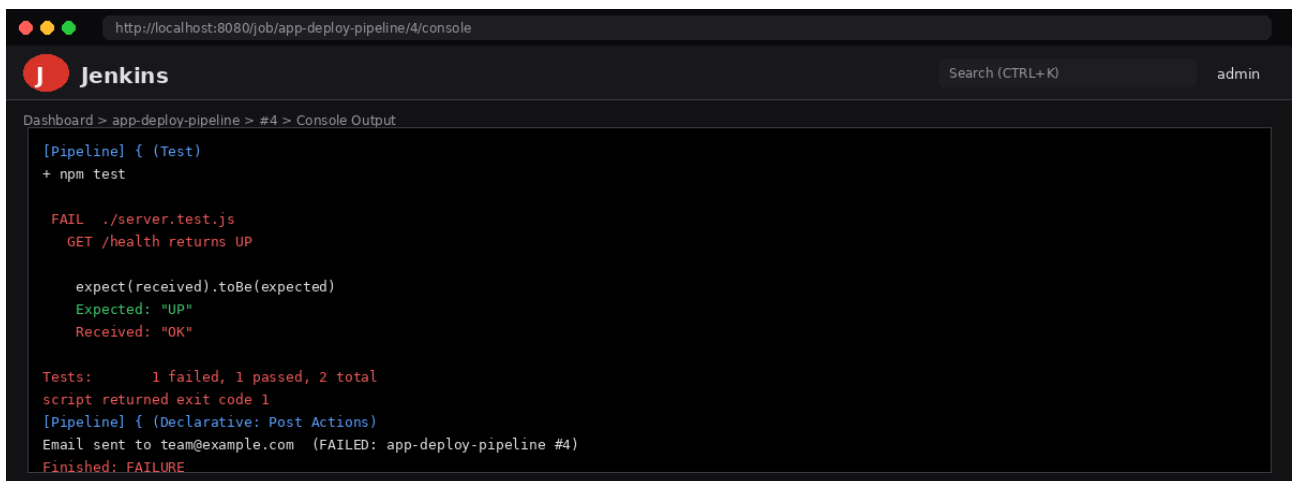
PASS ./server.test.js
  GET / returns greeting (38 ms)
  GET /health returns UP (12 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 1.482 s
Ran all test suites.
```

Figure 6: Jest test results in the Test stage (2 of 2 passed)

5.2 Failed run and recovery (build #4)

Build #4 failed in the Test stage because the /health endpoint returned OK instead of UP. The pipeline stopped before Build and Deploy, so the faulty code never reached the server, and a failure e-mail was sent. The bug was fixed and pushed, and build #5 passed.



```
http://localhost:8080/job/app-deploy-pipeline/4/console
Jenkins
Search (CTRL+K) admin
Dashboard > app-deploy-pipeline > #4 > Console Output

[Pipeline] { (Test)
+ npm test

FAIL ./server.test.js
  GET /health returns UP

    expect(received).toBe(expected)
    Expected: "UP"
    Received: "OK"

Tests: 1 failed, 1 passed, 2 total
script returned exit code 1
[Pipeline] { (Declarative: Post Actions)
Email sent to team@example.com (FAILED: app-deploy-pipeline #4)
Finished: FAILURE
```

Figure 7: Console output of failed build #4 - tests stopped the release



Figure 8: Build duration trend (red bar marks the failed build)

5.3 Summary metrics

Metric	Result
Total builds	7
Successful / failed	6 / 1
Build success rate	85.7%
Average duration (successful builds)	51 seconds
Latest build duration (#7)	48 seconds
Time from push to live application	about 1 minute (webhook to health check)
Manual deployment time (before automation)	about 15 minutes (estimated)
Recovery time after failed build #4	about 12 minutes (fix, push, rebuild)

6. Observations

Area	Observation
Speed	Push-to-live time dropped from about 15 minutes of manual work to about 1 minute.
Quality gate	The failing test in build #4 blocked the deployment, proving the Test stage protects the server.
Consistency	Every release used the same steps and a versioned artifact, so results were repeatable.
Triggers	Webhook builds started within seconds of a push; Poll SCM and cron acted as safety nets.
Feedback	Stage View and the failure e-mail pointed straight to the failing stage.
Resources	cleanWs() and build retention kept Jenkins disk usage low.
Limitation	There is a single target server, so a restart causes a few seconds of downtime.

7. Challenges and Solutions

Challenge	Solution
Webhook returned 403 at first	Enabled the GitHub plugin hook URL and allowed the Jenkins port in the firewall
ssh: Permission denied (publickey)	Copied the jenkins user public key to the target with ssh-copy-id and set the credential ID
Build #4 test failure	Corrected the /health response and added a test to prevent regression

8. Suggested Improvements

- Add a staging environment and deploy to production only after approval.
- Containerise the application with Docker for identical environments.
- Use blue-green or rolling deployment to remove downtime.
- Add code quality scanning (SonarQube) and dependency vulnerability checks.
- Send Slack or Teams notifications in addition to e-mail.

9. Conclusion

The CI/CD workflow automated the full path from a code push to a verified deployment. Testing gates prevented a faulty release, deployments became fast and repeatable, and the team received immediate feedback. Recording the results shows a high success rate and a large reduction in deployment time, and the suggested improvements provide a clear path towards a production-grade pipeline.